

Flash Payment APIs:	1
Access Token generation	1
Initiate a merchant payment	2
Retrieve Payment Order Details with Status	5
Refund a Payment Order	7
Cancel a Payment Order	8
Request payment on Instapay	10
Webhook API - Payment Transaction Notifications	11
HMAC Signature Calculations:	

Flash Payment APIs:

RESTful APIs are better suited for building our APIs due to their established standards, industry support, performance optimizations, and clear semantics.

The same API will be applied for Instapay or Flash. Only the difference is the integration ID, you will get a different integration ID based on your choice of payment.

Access Token generation

You should first authenticate to get an access token that will be required for any request to the API.

Base URL:

```
https://stg-api.useflash.app
```

Endpoint:

```
POST /v1/auth/token
```

Request Headers:

- **Content-Type:** application/json

- **Authorization:** Basic <base64_encoded_client_credentials>
- The client ID and secret are typically sent as part of the request headers using the Authorization header with the Basic authentication scheme. The client ID and secret are base64-encoded and combined as **client_id:client_secret**.

Response:

- Upon successful authentication, the API generates an **access_token** along with details such as the **token_type** (e.g., "Bearer") and **expires_in** indicating the token's expiration time.
- In case of authentication failure, the API responds with an appropriate error code and message, such as "invalid_client".

Success (HTTP Status Code: 200)

```
{
  "access_token": "<generated_access_token>",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

Error (HTTP Status Code: 401)

```
{
  "error": "INVALID_CLIENT",
  "error_description": "Invalid client credentials."
}
```

Initiate a merchant payment

Endpoint:

```
POST /v1/orders
```

Request Headers:

- **Content-Type:** application/json
- **Authorization:** Bearer <access_token>

-

Request Body:

Required:

- **integrationId**: Identifier for the merchant integration with Flash, Flash will share it with you.
- **aggregatorOrderId** (string, required): The merchant order id which will be used to identify the order on the merchant side, it should be unique too. [ex. Foodics Order ID]
- **amountCents** (number in **cents**, required): The amount to be paid.
- **currency** (string, required): The currency code capitalized

Optional:

- **validity** (number, optional): it should be in seconds. The default is 86400 for instapay, which means 1 day.
- **orderType** (selection, optional): The order type can be one of those values [pickup, delivery, dine-in, call_center, online].
Note: if the type is **call_center**, will send the link to the customer via sms or whatsapp, so in this case you should set the customer information, otherwise he will not receive the **phoneNumber**
- **webEnabled** (boolean, default false): If it was true, it means that the user will be redirected to a web page first and allows him to pay without the need to install Flash app.
- **terminalId** (string, optional): The terminal id is an identifier for the terminal that will show the QR, like **soundbox device id**. The merchant will be able to configure the terminal devices from his dashboard.
- **merchantName** (string, optional): A display name for the merchant.
- **branch** (string, optional): An identifier for the branch, it can be the branch Slug or ID. It helps to manage the transaction's visibility per branch on the dashboard of the merchant.
- **customer** (object, optional): Represent the customer info like name and phone number
- **notifiers**: list of notifier objects (**optional**) if included the courier will receive sms/whatsapp once the transaction is paid or any phone number you wanna send him a notification on mobile. It will have two fields
 - **phoneNumber**: the phone number that will receive the notification.
 - **type**: can be **sms** or **whatsapp** or **app-notification** [for app-notification, he should have an account with the same phone number on the courier app of Flash]
- **additionalInfo**: Array of key value pair objects, which can be used to attach metadata to order.

```
{
  "integrationId": 58328228,
  "aggregatorOrderId": "94fd204c-4f44-477a-8339-471cf6ed41f2",
  "description": "Order for product xyz",
  "customer": {
    "phone": "+20100456432222"
  },
}
```

```

"branch": "123467889",
"merchantName": "Lychee",
"terminalId": "cashier-1",
"webEnabled": true,
"amountCents": 5000,
"currency": "EGP",
"notifiers": [{
  "phoneNumber": "+201999999222",
  "type": "sms"
}],
"additionalInfo": [{"key": "product-code", "value": "lkmew"}]
}

```

Response:

Fields:

- **order** (object, required): Order object with the order details that have been added. The status of the created order will be **pending**, and after the user pays, it can be converted to **failed** or **succeeded**.
- **paymentLink** (url, required): This is the payment link that the user should use to pay and you can display it as QR for the user.

Notes:

- The API will always return a payment link even if the phone number isn't registered, so the user can scan the QR to download the app and signup and directly pay through the app.
The API handles various formats of phone numbers (e.g., with or without country codes, and specific formatting rules) to ensure compatibility
- The API validates the request and generates a **unique order ID** for the scan & pay process.
- In case of an error, the response includes an appropriate error code and message, providing details about the issue.
- For Instapay the **paymentLink** will be the instapay QR payload.

Success (HTTP Status Code: 201)

```

{
  "order": {
    "id": "string",
    "merchantId": "string",
    "aggregatorOrderId": "string",

```

```

    "billingInfo": {
      "customerName": "string",
      "phoneNumber": "string"
    },
    "amountCents": 100,
    "createdAt": "2023-08-06T18:14:55.230Z",
    "updatedAt": "2023-08-06T18:14:55.230Z",
    "status": "pending",
    "additionalInfo": [{"key": "product-code", "value": "lkmew"}]
  },
  "paymentLink": "https://pay.useflash.app/JSXET",
}

```

Expected Error Codes:

1. **EMPTY_AGGREGATOR_ORDER_ID**: when the request aggregatorOrderId is empty.
2. **EMPTY_AMOUNT**: when the request amountCents is empty.
3. **DUPLICATE_ORDER**: when an order with the same aggregatorOrderId already exists.
4. **MERCHANT_NOT_FOUND**: when a wrong merchantId is sent.
5. **INTEGRATION_NOT_FOUND**: When there is no integration with the entered merchantId.
5. **ORDER_BELOW_MINIMUM**: When the amountCents is less than 5 EGP.

Error Response Sample

```

{
  "error": {
    "code": "INTEGRATION_NOT_FOUND",
    "message": "We didn't find a matched integrationId"
  }
}

```

Retrieve Payment Order Details with Status

Endpoint:

```
GET /v1/orders/aggregator/{aggregatorOrderId}
```

Request Headers:

- **Content-Type**: application/json
- **Authorization**: Bearer <access_token>

Request Parameters: [URL parameter]

aggregatorOrderId (string, required): The identifier of the merchant order to fetch.

Response:

- The API endpoint accepts an orderId to identify the specific order to fetch
- If the order is found, the API responds with the order details, including the orderId, amount, currency, status, timestamp indicating when the order occurred, and customer details such as the name, and phone.
- In case of an error, such as when the order is not found, the API responds with an appropriate error code and message, indicating the issue.

Success (HTTP Status Code: 200)

```
{
  "id": "94fd204c-4f44-477a-8339-471cf6ed41f2",
  "aggregatorOrderId": "94fd204c-4f44-477a-8339-471cf6ed4w235",
  "amountCents": 5000,
  "currency": "EGP",
  "status": "pending",
  "createdAt": "2023-06-11T12:34:56Z",
  "billingInfo": {
    "customerName": "Ahmed Mostafa",
    "phoneNumber": "+201005398662"
  }
}
```

Error (HTTP Status Code: 404)

```
{
  "error": {
    "code": "ORDER_NOT_FOUND",
    "message": "The requested order could not be found."
  }
}
```

Expected Error Codes:

1. **EMPTY_AGGREGATOR_ORDER_ID**: when the request aggregatorOrderId is empty.
2. **ORDER_NOT_FOUND**: when the request aggregatorOrderId is empty.a

Refund a Payment Order

Endpoint:

```
POST /v1/orders/refund
```

Request Headers:

- **Content-Type:** application/json
- **Authorization:** Bearer <access_token>

Request Body:

- **orderId** (string, optional): The identifier of the original flash order ID being refunded.
- **merchantOrderId** (string, optional): The identifier of the merchant order ID being refunded.
- **amountCents** (number in **cents**, optional): The amount to be refunded.
- **Currency** (string, required): The currency code capitalized
- **reason** (string, optional): An optional reason or description for the refund.

You should provide at least one of the **orderId** or **merchantOrderId**. Otherwise we will not accept the refund request

```
{
  "orderId": "94fd204c-4f44-477a-8339-471cf6ed41f2",
  "amountCents": 5000,
  "currency": "EGP",
  "reason": "Customer requested a refund"
}
```

Response:

- Upon receiving a refund request, the API verifies the order ID and amount.
- If the refund is valid, it processes the refund and generates a unique refund_id.
- The response includes the refundId, orderId, amount refunded, status, and a timestamp indicating when the refund was processed.
- In case of an error, the response includes an appropriate error code and message, providing details about the issue.

Success (HTTP Status Code: 200)

```
{
  "refundId": "93d9e076-8ed5-4d69-b8b3-9b00e5efea52",
  "orderId": "94fd204c-4f44-477a-8339-471cf6ed41f2",
  "amountCents": 5000,
```

```
"currency": "EGP",
"status": "refunded",
"createdAt": "2023-06-11T12:34:56Z"
}
```

Error (HTTP Status Code: 400/500)

- Status code 400 in case of bad requests
- 500 for unexpected errors

```
{
  "error": {
    "code": "INVALID_AMOUNT",
    "message": "The refund amount exceeds the original order amount."
  }
}
```

Expected Error Codes:

1. **EMPTY_ORDER_ID**: when the request aggregatorOrderId is empty.
2. **PAYMENT_NOT_FOUND**: when nobody tried to pay the order.
3. **Transaction_Invalid_Status**: when the order payment status isn't successful.
4. **ORDER_NOT_FOUND**: when the request aggregatorOrderId doesn't exist.
5. **INVALID_REFUND_AMOUNT** When you send amount greater than the available amount to refund
6. **ALREADY_REFUNDED** When the order has been refunded before with full amount.
- 7.

Cancel a Payment Order

Endpoint:

```
POST /v1/orders/cancel
```

Request Headers:

- **Content-Type**: application/json
- **Authorization**: Bearer <access_token>

Request Body:

- **orderId** (string, optional): The identifier of the original flash order ID being canceled.
- **merchantOrderId** (string, optional): The identifier of the merchant order ID being canceled.
- **reason** (string, optional): An optional reason or description for the cancellation.

You should provide at least one of the **orderId** or **merchantOrderId**. Otherwise we will not accept the cancel request


```
{
  "merchantOrderId": "94fd204c-4f44-477a-8339-471cf6ed41f2",
  "reason": "Customer requested a cancel"
}
```

Response:

- Upon receiving a cancel request, the API validates the order ID and order status.
- If the order is still **pending** and the user didn't pay it, it processes the cancellation.
- In case of an error, the response includes an appropriate error code and message, providing details about the issue.

Success (HTTP Status Code: 200)

```
{
  "orderId": "94fd204c-4f44-477a-8339-471cf6ed41f2",
  "amountCents": 5000,
  "currency": "EGP",
  "status": "canceled",
  "createdAt": "2023-06-11T12:34:56Z"
}
```

Error (HTTP Status Code: 400/500)

- Status code 400 in case of bad requests
- 500 for unexpected errors

```
{
  "error": {
    "code": "EMPTY_ORDER_ID",
    "message": "please provide at least one of orderId or merchantOrderId"
  }
}
```

Expected Error Codes:

1. **EMPTY_ORDER_ID**: when the request merchantOrderId is empty.
3. **ORDER_ALREADY_PAID**: when the order payment status is successful or refunded.
4. **INVALID_STATUS**: The order status isn't pending, so we can't cancel it.

Request payment on Instapay

Endpoint:

POST /v1/orders/instapay/request-pay

Request Headers:

- **Content-Type:** application/json
- **Authorization:** Bearer <access_token>

Request Body:

- **customer** (object, required): Represent the customer info like name and phone number
- **aggregatorOrderId** (string, optional): The identifier of the merchant order ID.
- **amountCents** (number in **cents**, required): The amount to be paid.
- **currency** (string, required): The currency code capitalized
- **integrationId**: Identifier for the merchant integration with Flash, Flash will share it with you.

Optional:

- **validity** (number, optional): it should be in seconds. The default is 86400 for instapay, which means 1 day.
- **orderType** (selection, optional): The order type can be one of those values [pickup, delivery, dine-in, call-center].
Note: if the type is **call-center**, will send the link to the customer via sms or whatsapp, so in this case you should set the customer information, otherwise he will not receive the phoneNumber
- **merchantName** (string, optional): A display name for the merchant.
- **branch** (string, optional): An identifier for the branch, it can be the branch Slug or ID. It helps to manage the transaction's visibility per branch on the dashboard of the merchant.

```
{
  "integrationId": 58328228,
  "aggregatorOrderId": "94fd204c-4f44-477a-8339-471cf6ed41f2",
  "customer": {
    "phone": "+20100456432222"
  },
  "branch": "123467889",
  "merchantName": "Lychee",
  "amountCents": 5000,
  "currency": "EGP",
}
```

Response:

Success (HTTP Status Code: 200)

```
{
  "orderId": "94fd204c-4f44-477a-8339-471cf6ed41f2",
  "amountCents": 5000,
  "currency": "EGP",
  "status": "pending",
  "createdAt": "2023-06-11T12:34:56Z"
}
```

Webhook API - Payment Transaction Notifications

The Payment Transaction Webhook API allows you to receive real-time notifications about payment transactions on your platform.

Endpoint:

```
URL: https://www.example.com/webhooks/transaction-callback
Method: POST
```

The merchant should share with us the callbacks URL. The default signing is HMAC-SHA256.
The signing secret will be shared with the merchant

Request Headers:

Content-Type: application/json

signature: HMAC-SHA256 hash of the payload using the shared secret key

Request Payload:

The webhook request will include a JSON payload containing the payment transaction details. The payload format is as follows:

```
{
  "integrationId": 6741,
  "channelId": 61,
  "transactionId": "payment_transaction_id",
  "aggregatorOrderId": "merchant_order_id",
  "order": {
    "id": "flash_order_id",
```

```

    "description": "Order for product xyz",
    "customer": {
      "name": "customer name",
      "phone": "+20100456432222",
    },
    "amountCents": 5000,
    "currency": "EGP",
    "paymentLink": "https://pay.useflash.app/JSXET",
    "createdAt": "1735930920",
    "AdditionalInfo": {"trackingNumber": "1234456677"},
  },
  "PaidAmountCents": 5000,
  "status": "succeeded",
  "updatedAt": "1735930929"
}

```

Expected transaction status:

1. "pending": A transaction in the pending status indicates that it has been created and is currently awaiting processing.
2. "processing": This status signifies that the transaction is actively being handled and processed by the system.
3. "failed": When a transaction is marked as failed, it means it was unsuccessful or encountered an error during processing.
4. "succeeded": A transaction in the succeeded status denotes that it has been completed successfully.
5. "refunded": In cases where the amount of the transaction has been returned to the user, the status is labeled as refunded.
6. "canceled": This status indicates that the transaction was intentionally canceled. This can occur if the user fails to pass or cancels the 2FA verification step on the mobile app.

HMAC Signature Calculations:

HMAC is commonly used for ensuring data integrity and authentication by generating a cryptographic signature based on a shared secret key.

Try to do it in a generic way to support any json payload to avoid breaking the signature, if Flash added a new field to the payload.

Step 1: Prepare the JSON Object

The JSON object may contain simple key-value pairs as well as nested objects. The structure should be flattened into a string format for signature calculation. Flatten nested objects into a single-level structure to maintain consistency in generating the signature please check how it should look like.

```

{
  "integrationId":6741,

```

```

"transactionId":"payment_transaction_id",
"aggregatorOrderId":"merchant_order_id",
"order.id":"flash_order_id",
"order.description":"Order for product xyz",
"order.customer.name":"customer name",
"order.customer.phone":"+20100456432222",
"order.amountCents":5000,
"order.currency":"EGP",
"order.paymentLink":"https://pay.useflash.app/JSXET",
"order.createdAt":"1735930920",
"status":"succeeded",
"updatedAt":"1735930929"
}

```

Step 2: Omit any empty keys

The flatten JSON object shouldn't have any empty values. You should remove such keys if they exist.

Step 3: Sort the keys alphabetically

The flatten JSON object shouldn't have any empty values. You should remove such keys if they exist.

Before generating the signature, ensure that the keys in the JSON object are sorted. This ensures consistency in the signature calculation process. Sorting keys alphabetically is a common practice.

Step 4: Convert it into KV pairs string

Once the object is flattened and the keys are sorted, concatenate the key-value pairs into a string. The string should use a consistent format, such as key1=value1,key2=value2

```

"aggregatorOrderId=merchant_order_id,integrationId=6741,order.amountCents=5000,order.createdAt=1735930920,order.currency=EGP,order.customer.name=customer name,order.customer.phone="+20100456432222,order.description=Order for product xyz,order.id=flash_order_id,order.paymentLink=https://pay.useflash.app/JSXET,status=succeeded,transactionId=payment_transaction_id",updatedAt=1735930929

```

Step 5: Calculate the signature

Once you have the string ready, you can generate the HMAC signature using your preferred language to sign the output string with the secret key [*Flash will share it with you*] .

This is an example of how to do it in python.

```
signature = hmac.new(secret_key, payload_string.encode('utf-8'),  
hashlib.sha256).hexdigest()
```

Send Order Payment Link API

Overview

Sends a pending order payment link to a customer by SMS or WhatsApp. Currently this feature isn't enabled by default, if you wanna use it, you should ask us to turn it on.

Endpoint:

```
POST /v1/orders/send
```

Request Headers:

- **Content-Type:** application/json
- **Authorization:** Bearer <access_token>

Request Body:

- **orderId** (string): The identifier of the original flash order ID being canceled.
- **recipient** (string): Customer mobile number with country code, for example +2010XXXXXXXX.
- **channel** (string): Delivery channel. Allowed values: sms, whatsapp.

```
{
  "orderId": "550e8400-e29b-41d4-a716-446655440000",
  "recipient": "+2010XXXXXXXX",
  "channel": "sms"
}
```

Success (HTTP Status Code: 200)

```
{
  "orderId": "550e8400-e29b-41d4-a716-446655440000"
}
```

Error (HTTP Status Code: 400/500)

- Status code 400 in case of bad requests
- 500 for unexpected errors

```
{
  "error": {
    "code": "ORDER_SEND_ATTEMPTS_EXCEEDED",
    "message": "The refund amount exceeds the original order amount."
  }
}
```

Expected Error Codes:

ORDER_SEND_ATTEMPTS_EXCEEDED: Only we allow 3 attempts per order

